

# ECEN 662: Estimation & Detection Theory

## Lecture 20: Diffusion Monte Carlo Sampling

Tie Liu

Texas A&M University

# Brownian Motion and Diffusion

- *Diffusion Monte Carlo* utilizes a continuous-time stochastic process driven by *Brownian motion*.

# Brownian Motion and Diffusion

- *Diffusion Monte Carlo* utilizes a continuous-time stochastic process driven by *Brownian motion*.
- **Standard Brownian Motion**  $B = (B_t)_{t \geq 0}$ : A stochastic process satisfying:
  - *Initialization*:  $B_0 = 0$
  - *Gaussian Increments*: For any  $s, t \geq 0$ ,  $B_{s+t} - B_s \sim \mathcal{N}(0, t)$
  - *Independence*: Increments over non-overlapping intervals are independent.

# Brownian Motion and Diffusion

- *Diffusion Monte Carlo* utilizes a continuous-time stochastic process driven by *Brownian motion*.
- **Standard Brownian Motion**  $B = (B_t)_{t \geq 0}$ : A stochastic process satisfying:
  - *Initialization*:  $B_0 = 0$
  - *Gaussian Increments*: For any  $s, t \geq 0$ ,  $B_{s+t} - B_s \sim \mathcal{N}(0, t)$
  - *Independence*: Increments over non-overlapping intervals are independent.
- *Note*: In  $\mathbb{R}^d$ ,  $B_t$  consists of  $d$  independent 1D Brownian motions.

# Brownian Motion and Diffusion

- *Diffusion Monte Carlo* utilizes a continuous-time stochastic process driven by *Brownian motion*.
- **Standard Brownian Motion**  $B = (B_t)_{t \geq 0}$ : A stochastic process satisfying:
  - *Initialization*:  $B_0 = 0$
  - *Gaussian Increments*: For any  $s, t \geq 0$ ,  $B_{s+t} - B_s \sim \mathcal{N}(0, t)$
  - *Independence*: Increments over non-overlapping intervals are independent.
- *Note*: In  $\mathbb{R}^d$ ,  $B_t$  consists of  $d$  independent 1D Brownian motions.
- **General Diffusion Process**: Defined by the Stochastic Differential Equation (SDE):

$$\begin{aligned} X_0 &\sim p_0 \\ dX_t &= \underbrace{b(X_t, t)dt}_{\text{Drift}} + \underbrace{a(X_t, t)dB_t}_{\text{Diffusion}} \end{aligned}$$

where  $b : \mathbb{R}^d \times [0, \infty) \rightarrow \mathbb{R}^d$  and  $a : \mathbb{R}^d \times [0, \infty) \rightarrow \mathbb{R}^{d \times d}$ .

# Discretizing the SDE: Euler-Maruyama Method

- This is called a *stochastic differential equation (SDE)*, since the dynamics of  $X_t$  depend on the random process  $B_t$ .

# Discretizing the SDE: Euler-Maruyama Method

- This is called a *stochastic differential equation (SDE)*, since the dynamics of  $X_t$  depend on the random process  $B_t$ .
- Intuitively, the above equation means that for an (infinitesimally) small  $\Delta t > 0$ , we have:

$$X_{t+\Delta t} \approx X_t + b(X_t, t)\Delta t + a(X_t, t)(B_{t+\Delta t} - B_t)$$

# Discretizing the SDE: Euler-Maruyama Method

- This is called a *stochastic differential equation (SDE)*, since the dynamics of  $X_t$  depend on the random process  $B_t$ .
- Intuitively, the above equation means that for an (infinitesimally) small  $\Delta t > 0$ , we have:

$$X_{t+\Delta t} \approx X_t + b(X_t, t)\Delta t + a(X_t, t)(B_{t+\Delta t} - B_t)$$

- In other words, the conditional distribution for the next time step  $X_{t+\Delta t}$  given the current state  $X_t$  is:

$$X_{t+\Delta t} \mid X_t \sim \mathcal{N}(X_t + b(X_t, t)\Delta t, a(X_t, t)a^\top(X_t, t)\Delta t)$$

# Discretizing the SDE: Euler-Maruyama Method

- This is called a *stochastic differential equation (SDE)*, since the dynamics of  $X_t$  depend on the random process  $B_t$ .
- Intuitively, the above equation means that for an (infinitesimally) small  $\Delta t > 0$ , we have:

$$X_{t+\Delta t} \approx X_t + b(X_t, t)\Delta t + a(X_t, t)(B_{t+\Delta t} - B_t)$$

- In other words, the conditional distribution for the next time step  $X_{t+\Delta t}$  given the current state  $X_t$  is:

$$X_{t+\Delta t} \mid X_t \sim \mathcal{N}(X_t + b(X_t, t)\Delta t, a(X_t, t)a^\top(X_t, t)\Delta t)$$

- Therefore, the deterministic *drift*  $b$  describes the infinitesimal change in the mean, and the *volatility* function  $a$  describes the infinitesimal standard deviation.

# The Fokker-Planck Equation

- Let  $X_t \sim p_t$ . The **Fokker-Planck equation** (or Kolmogorov Forward Equation) describes how the *marginal* probability density  $p_t(x)$  evolves over time:

$$\begin{aligned} \frac{\partial}{\partial t} p_t(x) = & - \sum_{i=1}^d \frac{\partial}{\partial x_i} \left( b_i(x, t) p_t(x) \right) \\ & + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2}{\partial x_i \partial x_j} \left( (a(x, t) a^\top(x, t))_{i,j} p_t(x) \right) \end{aligned}$$

# The Fokker-Planck Equation

- Let  $X_t \sim p_t$ . The **Fokker-Planck** equation (or Kolmogorov Forward Equation) describes how the *marginal* probability density  $p_t(x)$  evolves over time:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) = & - \sum_{i=1}^d \frac{\partial}{\partial x_i} \left( b_i(x, t) p_t(x) \right) \\ & + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2}{\partial x_i \partial x_j} \left( (a(x, t) a^\top(x, t))_{i,j} p_t(x) \right)\end{aligned}$$

- Key Significance:** The change of the density  $p_t$  over *time*  $t$  can be expressed entirely using spatial derivatives over the *coordinates*  $x$ .

# The Fokker-Planck Equation

- Let  $X_t \sim p_t$ . The **Fokker-Planck** equation (or Kolmogorov Forward Equation) describes how the *marginal* probability density  $p_t(x)$  evolves over time:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) = & - \sum_{i=1}^d \frac{\partial}{\partial x_i} \left( b_i(x, t) p_t(x) \right) \\ & + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2}{\partial x_i \partial x_j} \left( (a(x, t) a^\top(x, t))_{i,j} p_t(x) \right)\end{aligned}$$

- Key Significance:** The change of the density  $p_t$  over *time*  $t$  can be expressed entirely using spatial derivatives over the *coordinates*  $x$ .
- Crucial Consequence:** Because the evolution of the density is deterministic, the marginal distributions of *every* SDE can also be achieved by simulating a deterministic *ordinary differential equation (ODE)*.

# Expressing an SDE via an ODE

- **The SDE Formulation:** Consider the *SDE* given by:

$$dX_t = b(X_t, t)dt + a(X_t, t)dB_t$$

# Expressing an SDE via an ODE

- **The SDE Formulation:** Consider the *SDE* given by:

$$dX_t = b(X_t, t)dt + a(X_t, t)dB_t$$

- **The ODE Question:** If we start with an initial distribution  $X_0 \sim p_0$ , does there exist a deterministic *ODE* of the form:

$$dZ_t = F(Z_t, t)dt$$

where  $Z_t$  and  $X_t$  share the exact same marginal distribution  $p_t$  for *all*  $t > 0$ ?

# Expressing an SDE via an ODE

- **The SDE Formulation:** Consider the *SDE* given by:

$$dX_t = b(X_t, t)dt + a(X_t, t)dB_t$$

- **The ODE Question:** If we start with an initial distribution  $X_0 \sim p_0$ , does there exist a deterministic *ODE* of the form:

$$dZ_t = F(Z_t, t)dt$$

where  $Z_t$  and  $X_t$  share the exact same marginal distribution  $p_t$  for *all*  $t > 0$ ?

- **The Probability Flow ODE:** The answer is (surprisingly) yes! The corresponding vector field  $F$  is given by:

$$F(x, t) = b(x, t) - \frac{1}{2}a(x, t)a^\top(x, t)\nabla_x \log p_t(x)$$

# Expressing an SDE via an ODE

- **The SDE Formulation:** Consider the *SDE* given by:

$$dX_t = b(X_t, t)dt + a(X_t, t)dB_t$$

- **The ODE Question:** If we start with an initial distribution  $X_0 \sim p_0$ , does there exist a deterministic *ODE* of the form:

$$dZ_t = F(Z_t, t)dt$$

where  $Z_t$  and  $X_t$  share the exact same marginal distribution  $p_t$  for *all*  $t > 0$ ?

- **The Probability Flow ODE:** The answer is (surprisingly) yes! The corresponding vector field  $F$  is given by:

$$F(x, t) = b(x, t) - \frac{1}{2}a(x, t)a^\top(x, t)\nabla_x \log p_t(x)$$

- **The Score Function:** The effect of a stochastic SDE can be achieved by running a deterministic ODE. However, to actually compute this, we must know the *score function*  $\nabla_x \log p_t(x)$ .

# Langevin Diffusion

- **The Langevin SDE:** In *Langevin diffusion*, the governing stochastic differential equation is:

$$dX_t = \nabla \log \pi(X_t)dt + \sqrt{2}dB_t$$

for some target distribution  $\pi$ .

# Langevin Diffusion

- **The Langevin SDE:** In *Langevin diffusion*, the governing stochastic differential equation is:

$$dX_t = \nabla \log \pi(X_t)dt + \sqrt{2}dB_t$$

for some target distribution  $\pi$ .

- **Stationary Distribution:** Through the Fokker-Planck equation, it can be mathematically shown that  $\pi$  is exactly the *stationary* distribution of this specific diffusion process.

# Langevin Diffusion

- **The Langevin SDE:** In *Langevin diffusion*, the governing stochastic differential equation is:

$$dX_t = \nabla \log \pi(X_t)dt + \sqrt{2}dB_t$$

for some target distribution  $\pi$ .

- **Stationary Distribution:** Through the Fokker-Planck equation, it can be mathematically shown that  $\pi$  is exactly the *stationary* distribution of this specific diffusion process.
- **Convergence Guarantee:** Furthermore, if  $\pi$  satisfies certain *functional inequalities* (like Log-Sobolev or Poincaré), the distribution of  $X_t$  converges *exponentially* to  $\pi$  as  $t \rightarrow \infty$ , regardless of the initial state  $X_0$ .

# Langevin Diffusion

- **The Langevin SDE:** In *Langevin diffusion*, the governing stochastic differential equation is:

$$dX_t = \nabla \log \pi(X_t)dt + \sqrt{2}dB_t$$

for some target distribution  $\pi$ .

- **Stationary Distribution:** Through the Fokker-Planck equation, it can be mathematically shown that  $\pi$  is exactly the *stationary* distribution of this specific diffusion process.
- **Convergence Guarantee:** Furthermore, if  $\pi$  satisfies certain *functional inequalities* (like Log-Sobolev or Poincaré), the distribution of  $X_t$  converges *exponentially* to  $\pi$  as  $t \rightarrow \infty$ , regardless of the initial state  $X_0$ .
- **The Score Function:** Therefore, through *Langevin diffusion*, we can sample from a target distribution  $\pi$  simply by simulating the SDE using its *score* function  $s(x) = \nabla_x \log \pi(x)$

# Langevin Diffusion

- **The Langevin SDE:** In *Langevin diffusion*, the governing stochastic differential equation is:

$$dX_t = \nabla \log \pi(X_t)dt + \sqrt{2}dB_t$$

for some target distribution  $\pi$ .

- **Stationary Distribution:** Through the Fokker-Planck equation, it can be mathematically shown that  $\pi$  is exactly the *stationary* distribution of this specific diffusion process.
- **Convergence Guarantee:** Furthermore, if  $\pi$  satisfies certain *functional inequalities* (like Log-Sobolev or Poincaré), the distribution of  $X_t$  converges *exponentially* to  $\pi$  as  $t \rightarrow \infty$ , regardless of the initial state  $X_0$ .
- **The Score Function:** Therefore, through *Langevin diffusion*, we can sample from a target distribution  $\pi$  simply by simulating the SDE using its *score* function  $s(x) = \nabla_x \log \pi(x)$
- **The Practical Advantage:** To compute the score function  $s(x)$ , we only need to know the target density  $\pi(x)$  up to an unknown normalizing constant!

# Naive Score Matching

- **The Learning Goal:** One way to learn a score-based model  $s_\theta(x)$  is by *matching* it to the true score function  $\nabla_x \log \pi(x)$  on a *per-sample* basis.

# Naive Score Matching

- **The Learning Goal:** One way to learn a score-based model  $s_\theta(x)$  is by *matching* it to the true score function  $\nabla_x \log \pi(x)$  on a *per-sample* basis.
- **The Objective Function:** Mathematically, this can be done by minimizing the *Fisher divergence*:

$$J_\pi(\theta) := \mathbb{E}_{X \sim \pi} [\|s_\theta(X) - \nabla_X \log \pi(X)\|^2]$$

# Naive Score Matching

- **The Learning Goal:** One way to learn a score-based model  $s_\theta(x)$  is by *matching* it to the true score function  $\nabla_x \log \pi(x)$  on a *per-sample* basis.
- **The Objective Function:** Mathematically, this can be done by minimizing the *Fisher divergence*:

$$J_\pi(\theta) := \mathbb{E}_{X \sim \pi} [\|s_\theta(X) - \nabla_X \log \pi(X)\|^2]$$

- **Hyvärinen's Trick:** While this seems *untenable* because  $\pi$  is *unknown*, a remarkable result by Hyvärinen (2005) uses integration by parts to show this is equivalent to minimizing:

$$\begin{aligned} \mathbb{E}_{X \sim \pi} \left[ \|s_\theta(X)\|^2 + 2 \sum_{j=1}^d \frac{\partial s_{\theta,j}(X)}{\partial X_j} \right] \\ \approx \frac{1}{m} \sum_{i=1}^m \left[ \|s_\theta(x[i])\|^2 + 2 \sum_{j=1}^d \frac{\partial s_{\theta,j}(x[i])}{\partial x_j} \right] \end{aligned}$$

# The Computational Bottleneck

- **The Trace Term:** Recall the critical term in the Hyvärinen objective:

$$\text{Tr}(\nabla_x s_\theta(x)) = \sum_{j=1}^d \frac{\partial s_{\theta,j}(x)}{\partial x_j}$$

# The Computational Bottleneck

- **The Trace Term:** Recall the critical term in the Hyvärinen objective:

$$\text{Tr}(\nabla_x s_\theta(x)) = \sum_{j=1}^d \frac{\partial s_{\theta,j}(x)}{\partial x_j}$$

- **The Jacobian Challenge:** To compute this trace exactly, we need the diagonal elements of the *Jacobian* matrix of our score model  $s_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ .
  - Standard backpropagation computes the gradient of a *scalar* loss.
  - Computing the full Jacobian diagonal requires  $d$  separate backward passes (one for each dimension).

# The Computational Bottleneck

- **The Trace Term:** Recall the critical term in the Hyvärinen objective:

$$\text{Tr}(\nabla_x s_\theta(x)) = \sum_{j=1}^d \frac{\partial s_{\theta,j}(x)}{\partial x_j}$$

- **The Jacobian Challenge:** To compute this trace exactly, we need the diagonal elements of the *Jacobian* matrix of our score model  $s_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ .
  - Standard backpropagation computes the gradient of a *scalar* loss.
  - Computing the full Jacobian diagonal requires  $d$  separate backward passes (one for each dimension).
- **High-Dimensional Scaling:** Consider a modern generative task:
  - For a  $1024 \times 1024$  RGB image,  $d \approx 3 \times 10^6$ .
  - Performing millions of backward passes *per training step* is computationally *prohibitive*.

# The Computational Bottleneck

- **The Trace Term:** Recall the critical term in the Hyvärinen objective:

$$\text{Tr}(\nabla_x s_\theta(x)) = \sum_{j=1}^d \frac{\partial s_{\theta,j}(x)}{\partial x_j}$$

- **The Jacobian Challenge:** To compute this trace exactly, we need the diagonal elements of the *Jacobian* matrix of our score model  $s_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ .
  - Standard backpropagation computes the gradient of a *scalar* loss.
  - Computing the full Jacobian diagonal requires  $d$  separate backward passes (one for each dimension).
- **High-Dimensional Scaling:** Consider a modern generative task:
  - For a  $1024 \times 1024$  RGB image,  $d \approx 3 \times 10^6$ .
  - Performing millions of backward passes *per training step* is computationally *prohibitive*.
- **The Path Forward:** To make Score Matching scalable, we must avoid the explicit computation of the Jacobian. This leads to two main families of methods: *Sliced Score Matching* and *Denoising Score Matching*.

# Sliced Score Matching (SSM)

- **The Core Idea:** To avoid computing the full Jacobian diagonal, we project the score onto a random direction  $v \sim p_v$  (where  $\mathbb{E}[vv^\top] = I$ ).

# Sliced Score Matching (SSM)

- **The Core Idea:** To avoid computing the full Jacobian diagonal, we project the score onto a random direction  $v \sim p_v$  (where  $\mathbb{E}[vv^\top] = I$ ).
- **The Sliced Fisher Divergence:** Instead of matching the full score vector, we match the *projections* of the scores:

$$J_{ssm}(\theta) := \mathbb{E}_{p_v} \mathbb{E}_{\pi} \left[ \frac{1}{2} \|v^\top s_\theta(X) - v^\top \nabla_X \log \pi(X)\|^2 \right]$$

# Sliced Score Matching (SSM)

- **The Core Idea:** To avoid computing the full Jacobian diagonal, we project the score onto a random direction  $v \sim p_v$  (where  $\mathbb{E}[vv^\top] = I$ ).
- **The Sliced Fisher Divergence:** Instead of matching the full score vector, we match the *projections* of the scores:

$$J_{ssm}(\theta) := \mathbb{E}_{p_v} \mathbb{E}_\pi \left[ \frac{1}{2} \|v^\top s_\theta(X) - v^\top \nabla_X \log \pi(X)\|^2 \right]$$

- **The Sliced Trace Trick:** Using integration by parts again, the objective becomes:

$$\mathbb{E}_{p_v} \mathbb{E}_\pi \left[ v^\top \nabla_X s_\theta(X) v + \frac{1}{2} \|v^\top s_\theta(X)\|^2 \right]$$

- The term  $v^\top \nabla_X s_\theta(X) v$  is a *Hessian-vector product*.
- This can be computed efficiently using forward-mode or reverse-mode AD in *one or two passes*.

# Sliced Score Matching (SSM)

- **The Core Idea:** To avoid computing the full Jacobian diagonal, we project the score onto a random direction  $v \sim p_v$  (where  $\mathbb{E}[vv^\top] = I$ ).
- **The Sliced Fisher Divergence:** Instead of matching the full score vector, we match the *projections* of the scores:

$$J_{ssm}(\theta) := \mathbb{E}_{p_v} \mathbb{E}_\pi \left[ \frac{1}{2} \|v^\top s_\theta(X) - v^\top \nabla_X \log \pi(X)\|^2 \right]$$

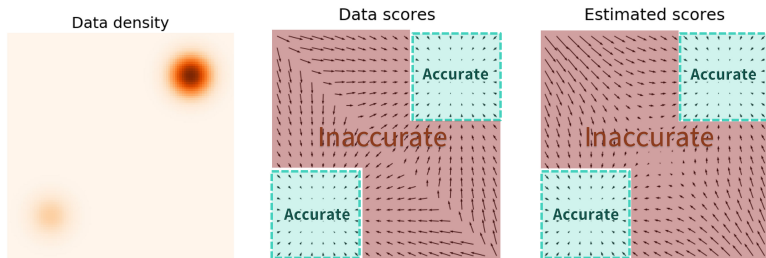
- **The Sliced Trace Trick:** Using integration by parts again, the objective becomes:

$$\mathbb{E}_{p_v} \mathbb{E}_\pi \left[ v^\top \nabla_X s_\theta(X) v + \frac{1}{2} \|v^\top s_\theta(X)\|^2 \right]$$

- The term  $v^\top \nabla_X s_\theta(X) v$  is a *Hessian-vector product*.
  - This can be computed efficiently using forward-mode or reverse-mode AD in *one or two passes*.
- **Efficiency:** We have reduced the complexity from  $O(d)$  backprops to  $O(1)$  (independent of the dimension  $d$ ).

# Pitfalls of Score Matching

- **The Data Sparsity Problem:** In practice, the aforementioned approaches have *limited* success. The main issue is that estimated score functions are inaccurate in *low-density* regions.
  - Score matching relies on samples to "see" the gradient.
  - Where there are *few* data points, the objective provides no signal to the model.



- **The Sampling Failure:** When sampling with Langevin dynamics, our initial sample (usually pure noise) is almost certain to start in a *low-density* region.
  - An inaccurate score model derails the dynamics from the *very beginning*.
  - The “gravity” of the score function points in random, wrong directions, preventing the model from ever reaching the high-quality data manifold.

- **Generalizing Sampling:** A more *general* idea for constructing a diffusion process is to use the *reversibility* of diffusion processes to sample from a target distribution.

# Reverse-time SDE

- **Generalizing Sampling:** A more *general* idea for constructing a diffusion process is to use the *reversibility* of diffusion processes to sample from a target distribution.
- **Forward Diffusion Process:** Consider the diffusion process described by the following SDE:

$$\begin{aligned} X_0 &\sim p_0 \quad (\text{Data distribution}) \\ dX_t &= b(X_t, t)dt + a(t)dB_t \end{aligned}$$

where we dropped the dependency of the velocity function  $a$  on  $X_t$  for simplicity.

# Reverse-time SDE

- **Generalizing Sampling:** A more *general* idea for constructing a diffusion process is to use the *reversibility* of diffusion processes to sample from a target distribution.
- **Forward Diffusion Process:** Consider the diffusion process described by the following SDE:

$$\begin{aligned} X_0 &\sim p_0 \quad (\text{Data distribution}) \\ dX_t &= b(X_t, t)dt + a(t)dB_t \end{aligned}$$

where we dropped the dependency of the velocity function  $a$  on  $X_t$  for simplicity.

- **Reverse-time SDE:** Consider the following *reverse-time* SDE (moving from  $T$  to 0):

$$\begin{aligned} \overline{X}_T &\sim p_T \quad (\text{Prior distribution}) \\ d\overline{X}_t &= \left( b(\overline{X}_t, t) - a^2(t) \nabla_{\overline{X}_t} \log p_t(\overline{X}_t) \right) dt + a(t) d\overline{W}_t \end{aligned}$$

where  $p_t$  is the marginal distribution of  $X_t$ , and  $\overline{W}_t$  is the *reverse-time* Brownian motion.

# Anderson's Reverser-time SDE Theorem

- **Theorem:** It was shown by Anderson (1982) that the reverse-time SDE generates the same marginal distributions as the forward process:

$$\overline{X}_t \sim p_t, \quad \forall t \in [0, T]$$

# Anderson's Reverser-time SDE Theorem

- **Theorem:** It was shown by Anderson (1982) that the reverse-time SDE generates the same marginal distributions as the forward process:

$$\overline{X}_t \sim p_t, \quad \forall t \in [0, T]$$

- **Generative Application:** To sample from a target distribution  $\pi$ :
  1. Define a forward SDE starting with  $X_0 \sim \pi$ .
  2. Choose a large  $T$  such that  $p_T$  is an *easy-to-sample-from* distribution (e.g., Gaussian noise).

# Anderson's Reverser-time SDE Theorem

- **Theorem:** It was shown by Anderson (1982) that the reverse-time SDE generates the same marginal distributions as the forward process:

$$\overline{X}_t \sim p_t, \quad \forall t \in [0, T]$$

- **Generative Application:** To sample from a target distribution  $\pi$ :
  1. Define a forward SDE starting with  $X_0 \sim \pi$ .
  2. Choose a large  $T$  such that  $p_T$  is an *easy-to-sample-from* distribution (e.g., Gaussian noise).
- **Generation Process:** The actual sampling is done by reversing the process:
  1. Sample  $x_T \sim p_T$ .
  2. Simulate the *reverse-time* SDE from  $t = T$  down to  $t = 0$ .

# Annealed Langevin Diffusion (1/2)

- **Forward Process:** Let the forward process be the *Ornstein–Uhlenbeck (OU)* process:

$$X_0 \sim \pi \quad (\text{Target data distribution})$$
$$dX_t = -\beta X_t dt + \sigma dW_t$$

with drift parameter  $\beta \geq 0$  and diffusion coefficient  $\sigma > 0$ .

# Annealed Langevin Diffusion (1/2)

- **Forward Process:** Let the forward process be the *Ornstein–Uhlenbeck (OU)* process:

$$\begin{aligned} X_0 &\sim \pi \quad (\text{Target data distribution}) \\ dX_t &= -\beta X_t dt + \sigma dW_t \end{aligned}$$

with drift parameter  $\beta \geq 0$  and diffusion coefficient  $\sigma > 0$ .

- **Data Destruction:** Since the transition probability is:

$$X_t|X_0 \sim \mathcal{N}\left(e^{-\beta t}X_0, \frac{\sigma^2}{2\beta}(1 - e^{-2\beta t})I\right)$$

for any  $t > 0$ ,  $X_T$  is *approximately* distributed as the stationary distribution:

$$\mathcal{N}\left(0, \frac{\sigma^2}{2\beta}I\right)$$

for sufficiently large  $T$ .

# Annealed Langevin Diffusion (1/2)

- **Forward Process:** Let the forward process be the *Ornstein-Uhlenbeck (OU)* process:

$$X_0 \sim \pi \quad (\text{Target data distribution})$$
$$dX_t = -\beta X_t dt + \sigma dW_t$$

with drift parameter  $\beta \geq 0$  and diffusion coefficient  $\sigma > 0$ .

- **Data Destruction:** Since the transition probability is:

$$X_t | X_0 \sim \mathcal{N} \left( e^{-\beta t} X_0, \frac{\sigma^2}{2\beta} (1 - e^{-2\beta t}) I \right)$$

for any  $t > 0$ ,  $X_T$  is *approximately* distributed as the stationary distribution:

$$\mathcal{N} \left( 0, \frac{\sigma^2}{2\beta} I \right)$$

for sufficiently large  $T$ .

- **Prior Distribution:** Sampling from  $\mathcal{N} \left( 0, \frac{\sigma^2}{2\beta} I \right)$  is *easy* (Gaussian noise).

# Annealed Langevin Diffusion (2/2)

- **Reverse-time SDE:** Using Anderson's theorem, the generative process is:

$$d\bar{X}_t = \left( -\beta \bar{X}_t - \sigma^2 \nabla_{\bar{X}_t} \log p_t(\bar{X}_t) \right) dt + \sigma d\bar{W}_t$$

# Annealed Langevin Diffusion (2/2)

- **Reverse-time SDE:** Using Anderson's theorem, the generative process is:

$$d\overline{X}_t = \left(-\beta\overline{X}_t - \sigma^2\nabla_{\overline{X}_t} \log p_t(\overline{X}_t)\right) dt + \sigma d\overline{W}_t$$

- **Sampling:** Starting from  $\overline{X}_T \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\beta}I\right)$ , simulating the reverse-time SDE yields  $\overline{X}_0 \approx \pi$ .

# Annealed Langevin Diffusion (2/2)

- **Reverse-time SDE:** Using Anderson's theorem, the generative process is:

$$d\bar{X}_t = \left(-\beta\bar{X}_t - \sigma^2\nabla_{\bar{X}_t} \log p_t(\bar{X}_t)\right) dt + \sigma d\bar{W}_t$$

- **Sampling:** Starting from  $\bar{X}_T \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\beta}I\right)$ , simulating the reverse-time SDE yields  $\bar{X}_0 \approx \pi$ .
- **Probability Flow ODE:** Alternatively, we can simulate the deterministic ODE:

$$d\bar{X}_t = \left(-\frac{\sigma^2}{2}\nabla_{\bar{X}_t} \log p_t(\bar{X}_t) - \beta\bar{X}_t\right) dt$$

# Annealed Langevin Diffusion (2/2)

- **Reverse-time SDE:** Using Anderson's theorem, the generative process is:

$$d\bar{X}_t = \left( -\beta \bar{X}_t - \sigma^2 \nabla_{\bar{X}_t} \log p_t(\bar{X}_t) \right) dt + \sigma d\bar{W}_t$$

- **Sampling:** Starting from  $\bar{X}_T \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\beta} I\right)$ , simulating the reverse-time SDE yields  $\bar{X}_0 \approx \pi$ .
- **Probability Flow ODE:** Alternatively, we can simulate the deterministic ODE:

$$d\bar{X}_t = \left( -\frac{\sigma^2}{2} \nabla_{\bar{X}_t} \log p_t(\bar{X}_t) - \beta \bar{X}_t \right) dt$$

- **Key Requirement:** Both reverse-time processes require the *noisy* score functions  $\nabla_x \log p_t(x)$ .

# Annealed Langevin Diffusion (2/2)

- **Reverse-time SDE:** Using Anderson's theorem, the generative process is:

$$d\bar{X}_t = (-\beta\bar{X}_t - \sigma^2 \nabla_{\bar{X}_t} \log p_t(\bar{X}_t)) dt + \sigma d\bar{W}_t$$

- **Sampling:** Starting from  $\bar{X}_T \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\beta} I\right)$ , simulating the reverse-time SDE yields  $\bar{X}_0 \approx \pi$ .
- **Probability Flow ODE:** Alternatively, we can simulate the deterministic ODE:

$$d\bar{X}_t = \left( -\frac{\sigma^2}{2} \nabla_{\bar{X}_t} \log p_t(\bar{X}_t) - \beta\bar{X}_t \right) dt$$

- **Key Requirement:** Both reverse-time processes require the *noisy* score functions  $\nabla_x \log p_t(x)$ .
- **Learning the Score:** These can be learned using *Implicit Score Matching* or *Denoising Score Matching* (Vincent, 2010).

# Perturbing Data with Noise

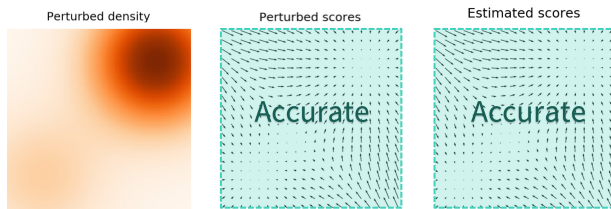
- **Addressing the Pitfall:** How can we bypass the difficulty of inaccurate score estimation in regions of low data density?

# Perturbing Data with Noise

- **Addressing the Pitfall:** How can we bypass the difficulty of inaccurate score estimation in regions of low data density?
- **The Noisy Solution:** A very clever approach is to *perturb* data points with noise and train score-based models on the *noisy* data points instead.

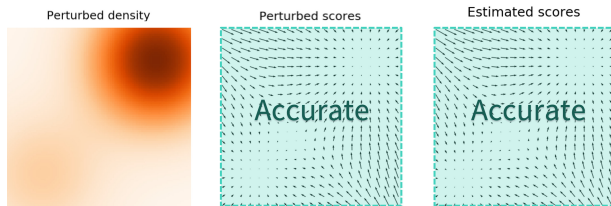
# Perturbing Data with Noise

- **Addressing the Pitfall:** How can we bypass the difficulty of inaccurate score estimation in regions of low data density?
- **The Noisy Solution:** A very clever approach is to *perturb* data points with noise and train score-based models on the *noisy* data points instead.
- **Filling the Void:** When the noise magnitude is sufficiently large, it populates low-density regions, providing the model with enough samples to estimate a reliable score field.



# Perturbing Data with Noise

- **Addressing the Pitfall:** How can we bypass the difficulty of inaccurate score estimation in regions of low data density?
- **The Noisy Solution:** A very clever approach is to *perturb* data points with noise and train score-based models on the *noisy* data points instead.
- **Filling the Void:** When the noise magnitude is sufficiently large, it populates low-density regions, providing the model with enough samples to estimate a reliable score field.



- **Key Question:** How do we choose an appropriate noise *scale* for the perturbation process? If it's too high, we lose the data; if it's too low, we're back to square one.

# Multi-scale Noise Perturbation

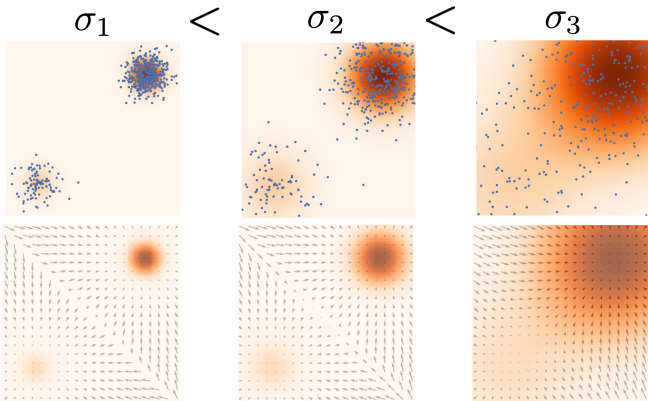
- **The High-Noise Trade-off:** Large noise covers more low-density regions for better global score estimation, but it *over-corrupts* the data, altering it significantly from the original distribution.

# Multi-scale Noise Perturbation

- **The High-Noise Trade-off:** Large noise covers more low-density regions for better global score estimation, but it *over-corrupts* the data, altering it significantly from the original distribution.
- **The Low-Noise Trade-off:** Small noise preserves the fine details of the original data distribution, but it fails to provide a helpful gradient in the distant, empty regions of the space.

# Multi-scale Noise Perturbation

- **The High-Noise Trade-off:** Large noise covers more low-density regions for better global score estimation, but it *over-corrupts* the data, altering it significantly from the original distribution.
- **The Low-Noise Trade-off:** Small noise preserves the fine details of the original data distribution, but it fails to provide a helpful gradient in the distant, empty regions of the space.
- **The Synthesis:** To achieve the best of both worlds, we use *multiple scales* of noise perturbations simultaneously. This allows the model to learn:
  - **Global structure** from high noise.
  - **Local details** from low noise.



# Diffusion GAN: The Density Ratio Perspective

- **The Modified ODE:** Alternatively, we can consider a specific *ODE* for sampling:

$$dX_t = \nabla_{X_t} \left( \log \frac{\pi(X_t)}{p_t(X_t)} \right) dt$$

# Diffusion GAN: The Density Ratio Perspective

- **The Modified ODE:** Alternatively, we can consider a specific *ODE* for sampling:

$$dX_t = \nabla_{X_t} \left( \log \frac{\pi(X_t)}{p_t(X_t)} \right) dt$$

- **Sampling via Density Ratios:** If we have access to the *density ratio*  $\pi(x)/p_t(x)$ , we can sample from the target distribution  $\pi$  by simulating the deterministic path of this ODE.

# Diffusion GAN: The Density Ratio Perspective

- **The Modified ODE:** Alternatively, we can consider a specific *ODE* for sampling:

$$dX_t = \nabla_{X_t} \left( \log \frac{\pi(X_t)}{p_t(X_t)} \right) dt$$

- **Sampling via Density Ratios:** If we have access to the *density ratio*  $\pi(x)/p_t(x)$ , we can sample from the target distribution  $\pi$  by simulating the deterministic path of this ODE.
- **Variational Characterization:** We can learn this ratio through an adversarial objective. The optimal discriminator  $d^*(x)$  satisfies:

$$d^*(x) = \frac{\pi(x)}{\pi(x) + p_t(x)}$$

$$d^*(x) = \arg \max_{d: \mathcal{X} \rightarrow [0,1]} \left( \mathbb{E}_{X \sim \pi} [\log d(X)] + \mathbb{E}_{X \sim p_t} [\log(1 - d(X))] \right)$$

# Diffusion GAN: The Density Ratio Perspective

- **The Modified ODE:** Alternatively, we can consider a specific *ODE* for sampling:

$$dX_t = \nabla_{X_t} \left( \log \frac{\pi(X_t)}{p_t(X_t)} \right) dt$$

- **Sampling via Density Ratios:** If we have access to the *density ratio*  $\pi(x)/p_t(x)$ , we can sample from the target distribution  $\pi$  by simulating the deterministic path of this ODE.
- **Variational Characterization:** We can learn this ratio through an adversarial objective. The optimal discriminator  $d^*(x)$  satisfies:

$$d^*(x) = \frac{\pi(x)}{\pi(x) + p_t(x)}$$

$$d^*(x) = \arg \max_{d: \mathcal{X} \rightarrow [0,1]} \left( \mathbb{E}_{X \sim \pi} [\log d(X)] + \mathbb{E}_{X \sim p_t} [\log(1 - d(X))] \right)$$

- **Discriminator-Guided Steps:** We train a *discriminator*  $d_\theta(x, t)$  using samples from  $\pi$  and the current model  $p_t$ , then use its gradients to simulate *one step* of the ODE.